



# The Browser Object Model

A Developer's Blueprint for  
Window, State, and Navigation APIs

A complete architectural breakdown of JavaScript's environment wrappers.



## DOM - Document Object Model

**Root:** `document`

**Focus:** The HTML document content and node tree.

**Standard:** W3C Official Standard.

**Examples:** `getElementById`,  
`querySelector`

## BOM - Browser Object Model

**Root:** `window` (The Global Wrapper)

**Focus:** The browser window, tab, and system environment.

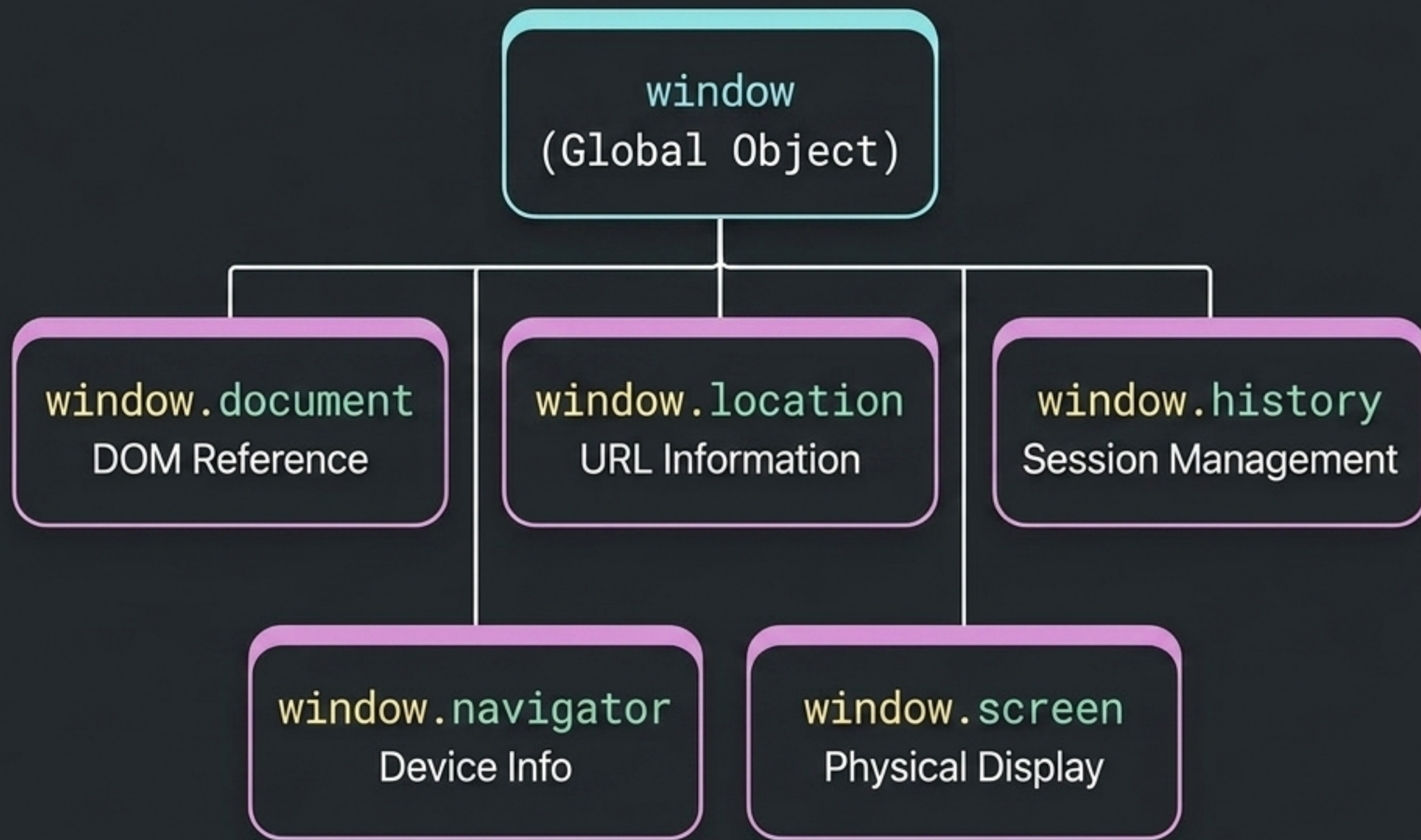
**Standard:** No official standard (de facto consistent across browsers).

**Examples:** `location`, `history`,  
`navigator`

```
// Every global variable, function, and object in JavaScript
// is actually a property of the window object.
window.alert() === alert(); // true
```



# The Universal Wrapper: `window`



## ⚠ Node.js Distinction

In Node.js, the global object is `global`, not `window`.

Code relying on `window` will break in backend environments.

### Tip

Because `window` is global, the prefix can be omitted.

`window.alert()` and `alert()` are identical.



# Spatial Anatomy: Viewport vs. Screen

## Coordinate Methods

`window.scrollX` / `scrollY`  
Pixels scrolled from origin.

`window.scrollTo(x,y)` /  
`scrollBy(x,y)`  
Absolute vs. relative scroll.

`window.devicePixelRatio`  
CSS pixels to physical pixels  
(e.g., 2 on Retina displays).

`screen.width` (1920px)

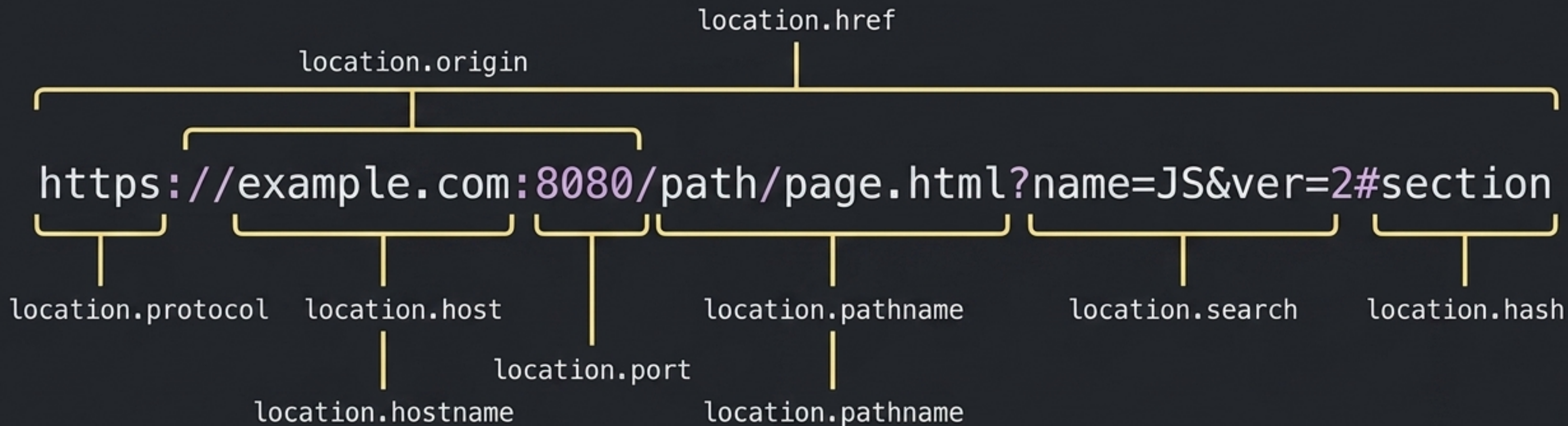
`screen.availWidth` (Minus OS Taskbar)

`window.outerWidth` (Entire browser chrome)

`window.innerWidth` (Viewport)



# Dissecting the URL: `window.location`



The `location.search` property returns the raw query string. Wrap it in `new URLSearchParams()` to parse and extract individual parameters.



# Execution & Routing: Navigation Methods

## The Standard Hop

`location.assign(url)`

Loads a new URL and adds it to the browser's history stack. The user can click 'Back'.

Equivalent to modifying `location.href`

## The One-Way Trip

`location.replace(url)`

Loads a new URL but *replaces* the current entry in history.

Crucial for login redirects so users cannot press 'Back' into a login form.

## The Refresh

`location.reload()`

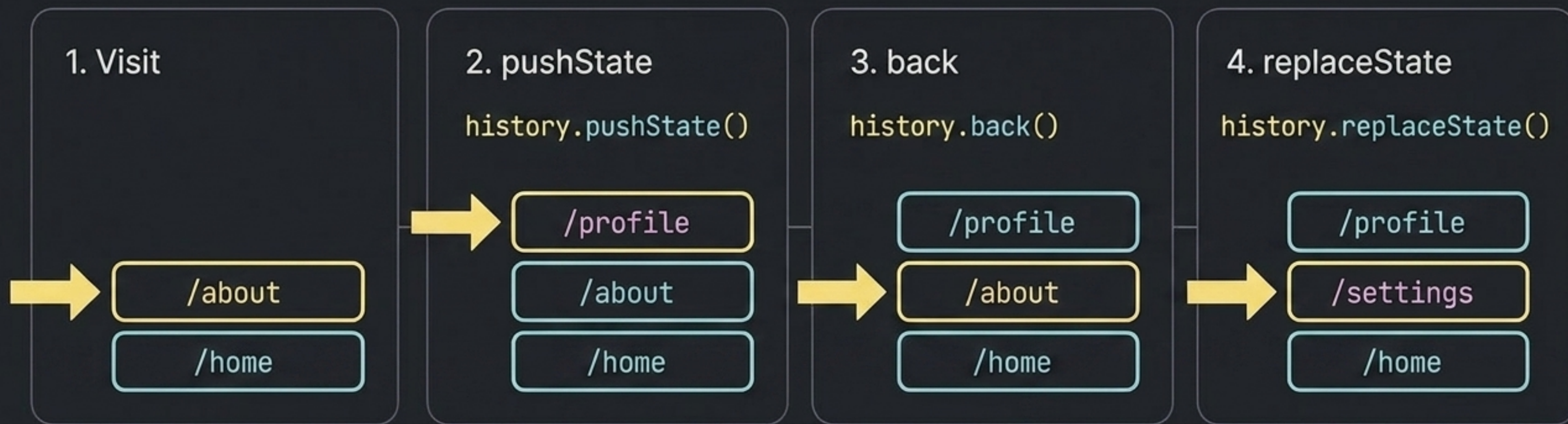
Reloads the current page, fetching from cache by default.

## Window Management

Use `window.open(url, target)` to open new tabs. The `window.close()` method only works on scripts that opened the tab.



# Time Travel & SPA State: `window.history`



(No page reload occurs)

## Event Listener

Listen for history changes using

```
window.addEventListener('popstate', handler)
```



# Hardware & Environment Profiling:

`window.navigator`

## Network & Locale

`navigator.onLine`  
(boolean status)

`navigator.languages`  
(user preferred locales)

## Hardware Probing

`navigator.hardwareConcurrency`  
(logical CPU cores)

`navigator.maxTouchPoints`  
(simultaneous touch capability)

## System Identity

`navigator.platform`  
(OS string, e.g., 'MacIntel')

`navigator.vendor`  
(e.g., 'Google Inc.')

 **Warning:** `navigator.userAgent`

The User-Agent string is highly unreliable and frequently spoofed by browsers. Do not use it for browser detection. Always prefer feature detection instead (e.g., `if ('geolocation' in navigator)`).



# Bridging the Gap: Native Device APIs



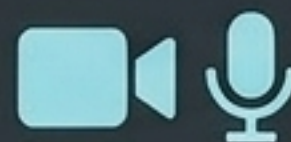
`navigator.geolocation`

Access GPS/location via  
`getCurrentPosition`.



`navigator.clipboard`

Modern async text/data  
read/write.



`navigator.mediaDevices`

Access camera and microphone  
via `getUserMedia`.



`navigator.serviceWorker`

Manage background scripts for  
PWAs.



`navigator.sendBeacon()`

Reliably send analytics data during  
page unload without blocking.



`navigator.vibrate()` / `share()`

Trigger device hardware vibration  
and native share sheets.

 Note: `geolocation`, `clipboard`, and `mediaDevices` require explicit user permission via the `navigator.permissions` API.



# Synchronous Interruptions: BOM Dialogs

```
// Main execution thread
function main.thread() {
  const arc = moint.create({ message: 'Critical Update Available' });
  console.logIntln("Critical update available");
};
```

Critical Update Available

OK

alert(message) -> void

Displays a message.

```
// main execution thread
const message = "New message entered";
function.siole('default') {
  // alert.println("message");
  output.this.confirm(true);
  console.logComplete('message', 'Are you sure you want to delete this item?');
};
```

Are you sure you want to delete this item?

Cancel

OK

confirm(message) -> boolean

Returns true for OK, false for Cancel.

Enter your name:

John Doe

Cancel

OK

prompt(message, default) -> string | null

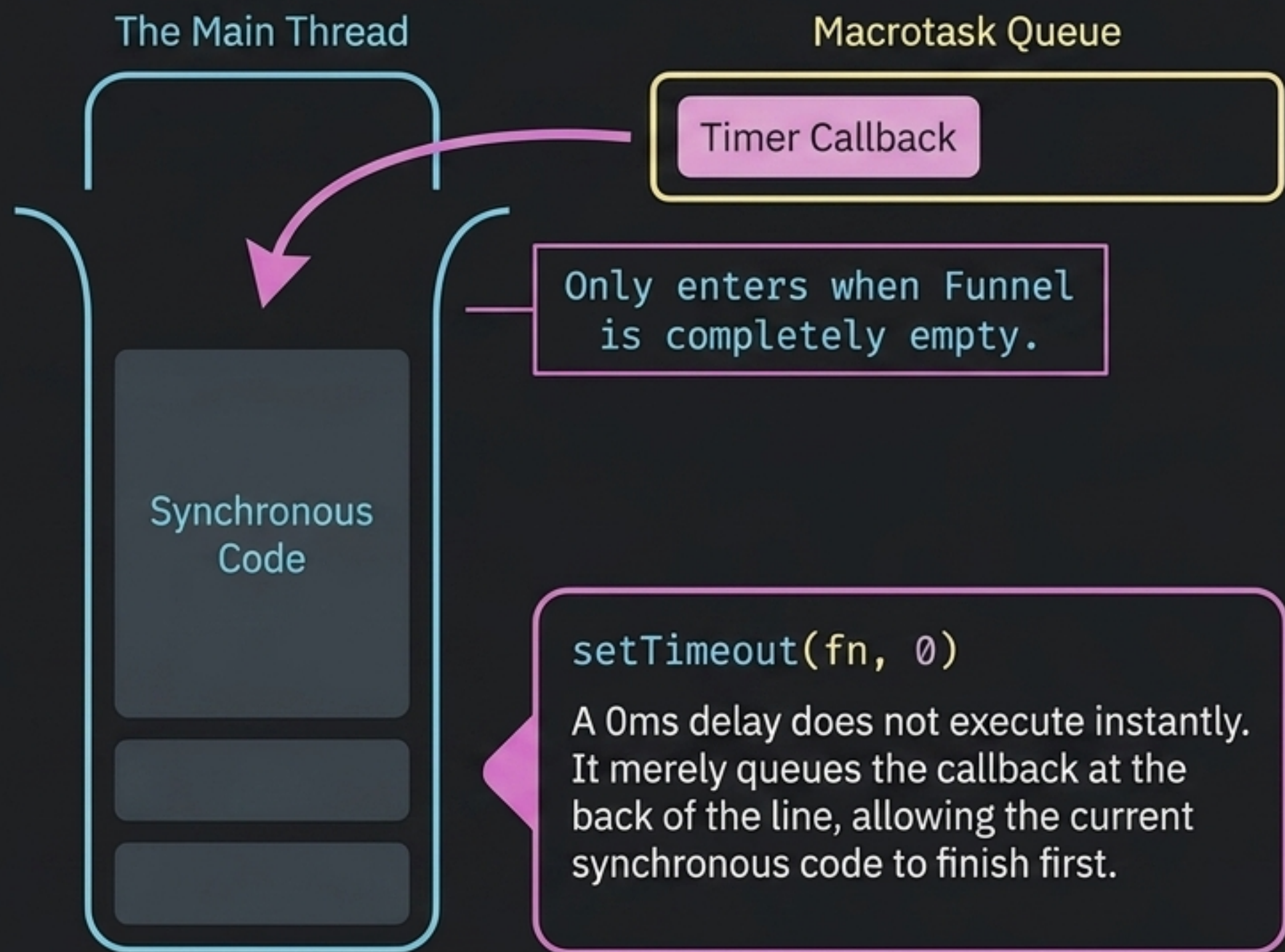
Returns the string entered or null.



**Critical Architecture Warning:** These methods block script execution and pause the DOM. Avoid them in modern production applications in favor of custom asynchronous modal dialogs.



# Time Scheduling & The Event Loop



```
setTimeout(fn, ms) /  
clearTimeout(id)
```

Executes the function once after the specified delay.

```
setInterval(fn, ms) /  
clearInterval(id)
```

Executes repeatedly. (Tip: Can drift over time; use recursive `setTimeout` for precision).

```
requestAnimationFrame(fn)
```

Runs before the next repaint at ~60fps. The optimal choice for UI animations.



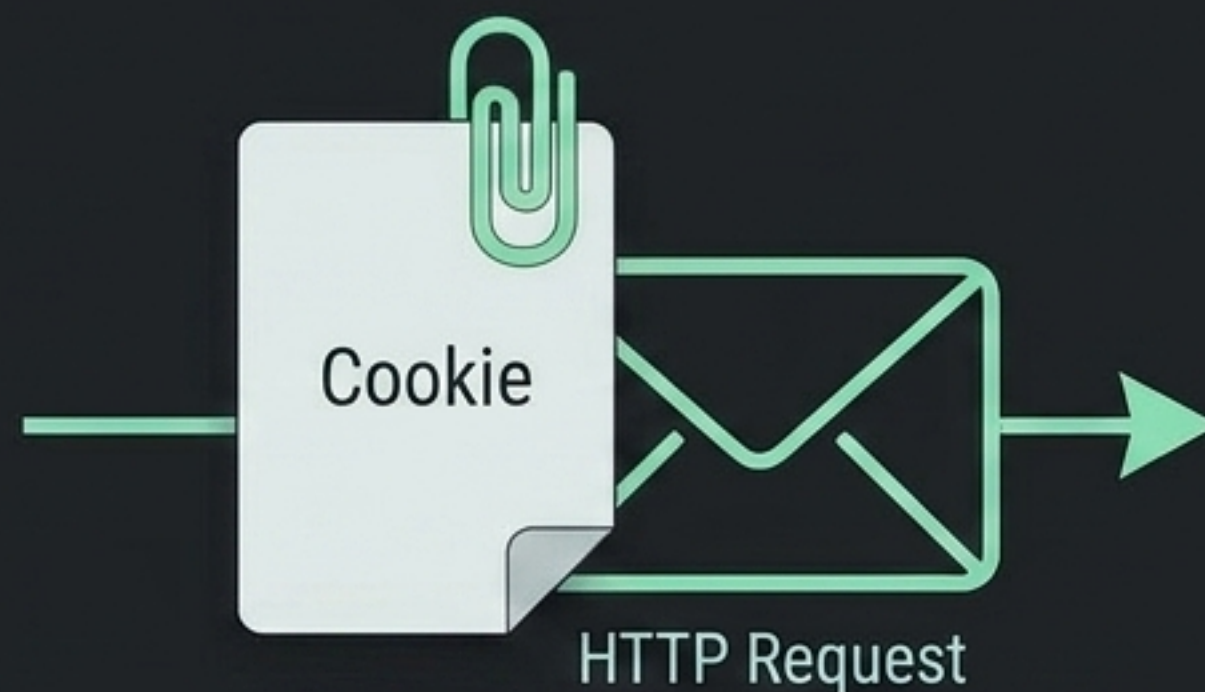
# The Legacy Network Memory: Cookies

`name=value; expires=date; path=/; HttpOnly; SameSite=Strict`

- **Payload:** `name=value` (max ~4KB total).
- **Lifetime:** `expires` or `max-age`. Without these, it's a transient Session Cookie.
- **Scope:** `path=/` and `domain=example.com`.



The `HttpOnly` flag renders the cookie completely invisible to JavaScript, protecting session tokens from Cross-Site Scripting (XSS) attacks.

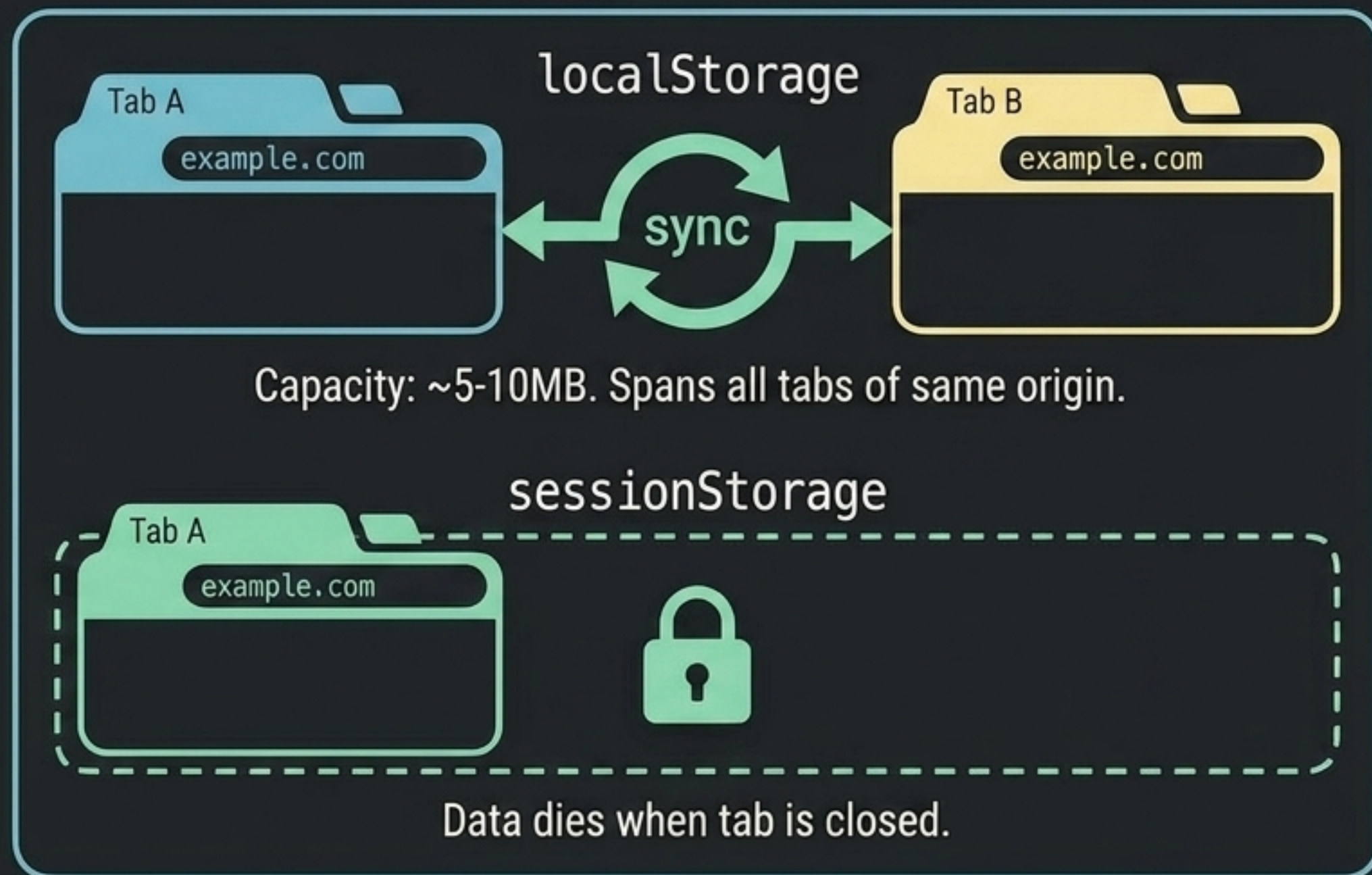


Cookies are automatically sent to the server on *\*every\** request, creating bandwidth overhead.



# Modern Client Memory: Web Storage API

## Tab Isolation



## API Methods

- `setItem(k, v)`
- `getItem(k)`
- `removeItem(k)`
- `clear()`

## Technical Constraints

- **Warning:** Web Storage only stores strings. Always use `JSON.stringify()` and `JSON.parse()`.
- **Event:** The `storage` event fires across tabs, enabling serverless communication.
- **Synchronous:** Blocks the main thread.



# The Storage Decision Matrix

| Feature           | Cookies               | localStorage       | sessionStorage       |
|-------------------|-----------------------|--------------------|----------------------|
| Capacity          | ~4 KB                 | ~5-10 MB           | ~5-10 MB             |
| Sent to Server    | Yes (Auto)            | No                 | No                   |
| Lifetime          | Expiry / Session      | Manually cleared   | Tab close            |
| Cross-Tab Scope   | Yes                   | Yes                | No (Same tab)        |
| Accessible via JS | Yes (Unless HttpOnly) | Yes                | Yes                  |
| Data Type         | Strings (Manual)      | Strings (JSON API) | Strings (JSON API)   |
| Primary Use Case  | Session IDs & Auth    | User preferences   | Temporary form state |

**Choose Cookies** for server-read authentication. Choose **LocalStorage** for persistent client data. Choose **SessionStorage** for temporary UI state.